

大模型性能质量指标体系——详细解释

本文档对 7 大类共 70 个大模型性能质量指标进行逐一详细解释，涵盖定义、计算方式、影响因素、优化方向和实际意义。

一、延迟类指标（12 个）

延迟类指标衡量模型响应速度，直接影响用户体验，是最常被关注的一类指标。

1. 首 Token 延迟（TTFT, Time To First Token）

项目	内容
缩写	TTFT
单位	毫秒（ms）
定义	从用户提交输入到模型生成第一个输出 token 所需的时间

详细解释：

TTFT 是交互式场景中用户感知最强烈的指标。当你在对话框中按下发送键后，到看到第一个字出现的那段"等待时间"，就是 TTFT。

计算方式：

Plain Text

1 TTFT = 请求排队时间 + Tokenize 时间 + Prefill 时间 + 第一个 token 的 Decode 时间

影响因素：

- 输入长度：prompt 越长，Prefill 越慢，TTFT 越大
- GPU 算力：Prefill 是计算密集型，算力越强越快
- 调度排队：高并发时请求可能排队等待 GPU 资源
- KV Cache 分配：大上下文需预分配大量 KV Cache 显存

优化方向：

- 使用 Flash Attention 加速注意力计算
- 采用 Prefix Caching 缓存公共 prompt 前缀
- 使用投机解码（Speculative Decoding）提前生成候选 token
- 多卡并行 Prefill（Tensor Parallelism）

实际参考值：

- 实时对话要求：TTFT < 200ms
- 可接受体验：TTFT < 500ms
- 长文档场景：TTFT 可达数秒

2. 每 Token 输出时间（TOPT, Time Of Per Token）

项目	内容
缩写	TOPT
单位	毫秒（ms）
定义	生成单个输出 token 所需的平均时间

详细解释：

TOPT 衡量的是 Decode 阶段每一步的速度。它和 Output TPS 是倒数关系： $TOPT = 1000 / \text{Output TPS}$ 。

计算方式：

Plain Text

1 $TOPT = \text{Decode 总时间} / \text{输出 token 数}$

影响因素：

- **模型参数量**：参数越多，每步需读取的权重越大
- **显存带宽**：Decode 是访存密集型，带宽是核心瓶颈
- **量化精度**：INT4 权重体积更小，读取更快，TOPT 更低
- **KV Cache 大小**：上下文越长，KV Cache 越大，注意力计算访存越多

优化方向：

- 量化（INT4/INT8）减少权重访存量
- 使用 Marlin 等高效 Decode 内核
- Flash Attention 减少注意力访存
- KV Cache 压缩（如 H2O、Scissorhands）

实际参考值（Llama-3-8B, RTX 4090）：

- FP16: TOPT $\approx$ 12-15ms（约 65-80 t/s）
- INT4 AWQ + Marlin: TOPT $\approx$ 1.3-2ms（约 500-741 t/s）

3. 相邻 Token 间隔（TBT, Time Between Tokens）

项目	内容
缩写	TBT
单位	毫秒（ms）
定义	相邻两个输出 token 之间的实际间隔时间

详细解释：

TBT 和 TOPT 的区别在于：TOPT 是平均值，TBT 关注每次间隔的实际值。如果 TBT 波动很大（如大部分时间 10ms，偶尔跳到 100ms），说明系统调度不稳定或存在资源抢占。

影响因素：

- GPU 调度抖动（其他请求抢占算力）
- KV Cache 换入换出
- 操作系统调度中断
- 功耗管理（GPU 降频）

优化方向：

- 绑核绑卡，减少调度干扰
- 实时调度策略（SCHED_FIFO）
- 预留足够显存避免换入换出

实际意义：

- TBT 稳定 → 用户感知流畅
- TBT 波动大 → 用户感知"卡顿"
- 实时字幕/语音场景对 TBT 稳定性极其敏感

4. 末 Token 延迟 (TTLT, Time To Last Token)

项目	内容
缩写	TTLT
单位	毫秒 (ms)
定义	从输入提交到生成最后一个输出 token 的时间

详细解释：

TTLT 是用户感知的"完整响应时间"——从发出请求到看到完整回答的总耗时。

计算方式：

Plain Text

1

TTLT = TTFT + Decode 总时间

2

TTLT = TTFT + (输出 token 数 × 平均 TOPT)

与 ITL 的关系：TTLT 和 ITL 在数值上等价，都是总推理耗时，但 TTLT 强调"最后一个 token 的时间点"，ITL 强调"总耗时"。

实际参考值：

- 短回答 (50 tokens)：TTLT ≈ TTFT + 0.5-1s
- 长回答 (500 tokens)：TTLT ≈ TTFT + 5-10s

5. 总推理耗时 (ITL, Inference Total Latency)

项目	内容
----	----

缩写	ITL
单位	毫秒（ms）
定义	从输入到生成所有输出 token 的总时间

详细解释：

ITL 与 TTLT 等价，是从不同角度描述同一指标。ITL 侧重于"过程的总时长"，TTLT 侧重于"最后一个 token 的时间点"。在不同框架和论文中可能使用不同名称。

计算方式：

Plain Text

1

ITL = 排队时间 + Prefill 时间 + Decode 时间

2

ITL = TTLT

6. 端到端延迟（E2E Latency）

项目	内容
缩写	E2E Latency
单位	毫秒（ms）
定义	从请求发出到完整响应接收的总耗时

详细解释：

E2E Latency 是最接近用户真实体验的指标，因为它包含了 ITL 没有计入的额外开销：

Plain Text

1

E2E Latency = 网络传输(请求) + 序列化 + 排队等待 + ITL + 反序列化 + 网络传输(响应)

与 ITL 的区别：

维度	ITL	E2E Latency
范围	纯推理耗时	包含网络、序列化等

场景	模型性能评估	用户体验评估
值	较小	较大（多出网络开销）

实际参考值：

- 同机房部署： $E2E \approx ITL + 5-20ms$
- 跨区域部署： $E2E \approx ITL + 50-200ms$
- 公网客户端： $E2E \approx ITL + 100-500ms$

7. Prefill 耗时

项目	内容
单位	毫秒（ms）
定义	处理输入 prompt（预填充 KV Cache）的耗时

详细解释：

Prefill 是推理的第一阶段，核心工作是：

1. 将所有输入 token 并行通过 Encoder/Decoder
2. 计算并缓存每一层的 Key 和 Value（KV Cache）
3. 为后续 Decode 阶段提供上下文

关键特性：

- **计算密集型**：所有输入 token 可并行处理，充分利用 GPU 算力
- **GPU 利用率高**：通常可达 80%+
- **与输入长度正相关**：近似线性关系
- **与输出无关**：Prefill 完成后才开始生成

计算量：

Plain Text

1 Prefill FLOPs $\approx 2 \times$ 参数量 $\times$ 输入 token 数

优化方向：

- Flash Attention 减少内存访问
- Tensor Parallelism 多卡并行
- Prefix Caching 缓存公共前缀
- Chunked Prefill 分块处理长 prompt

8. Decode 耗时

项目	内容
单位	毫秒（ms）
定义	自回归逐 token 生成阶段的总耗时

详细解释：

Decode 是推理的第二阶段，核心工作是：

- 将上一个 token 输入模型
- 读取全部模型权重和 KV Cache
- 计算下一个 token 的概率分布
- 采样得到下一个 token
- 更新 KV Cache
- 重复直到生成结束

关键特性：

- 访存密集型：**每步需读取全部权重 + KV Cache，但只做极少计算
- GPU 利用率低：**通常仅 5-20% ("内存带宽瓶颈")
- 与输出长度正相关：**输出越多 token，总时间越长
- 单步耗时稳定：**每步耗时近似相同（受 KV Cache 增长影响略增）

计算量：

Plain Text

1 单步 Decode FLOPs $\approx 2 \times$ 参数量

2 Decode 总 FLOPs $\approx 2 \times$ 参数量 $\times$ 输出 token 数

优化方向：

- 量化减少权重大小 → 减少访存量
- 高效 Decode 内核（Marlin、CUTLASS）
- Continuous Batching 提高并发利用率
- 投机解码（Speculative Decoding）

9-12. 延迟百分位数（P50 / P90 / P95 / P99）

项目	内容
缩写	P50 / P90 / P95 / P99
单位	毫秒（ms）
定义	分别表示 50%/90%/95%/99% 的请求延迟低于此值

详细解释：

百分位数是评估服务质量的统计方法。假设 1000 次请求的 TTFT 排序后：

- P50 = 200ms → 500 次请求的 TTFT < 200ms（中位数）
- P90 = 350ms → 900 次请求的 TTFT < 350ms
- P95 = 500ms → 950 次请求的 TTFT < 500ms
- P99 = 1200ms → 990 次请求的 TTFT < 1200ms

为什么不只看平均值？

- 平均值会被极端值拉偏
- 平均 300ms 但 P99 = 2s → 1% 的用户体验极差
- SLA 通常承诺 P99 或 P95，而非平均值

各百分位数的意义：

百分位	意义	用途
-----	----	----

P50	典型体验	日常监控
P90	大多数用户的体验下限	基本质量保证
P95	SLA 常用基准	服务承诺
P99	最差情况	关键场景保障

长尾产生的原因：

- 请求输入长度差异大
- 并发突发导致排队
- KV Cache 换入换出
- GPU 调度抖动

二、吞吐类指标（11 个）

吞吐类指标衡量模型的处理能力和系统承载力。

13. 输入 Token 数

项目	内容
单位	tokens
定义	输入给模型的文本长度（prompt 长度）

详细解释：

输入 token 数直接影响：

- Prefill 耗时（线性关系）
- KV Cache 占用（线性关系）
- TTFT（正相关）

注意事项：

- 不同 tokenizer 对同一文本的 token 数不同
- 中文 token 效率通常低于英文（1 个汉字 $\approx$ 1.5-3 tokens）

- 特殊 token（BOS、EOS、system prompt）也计入
- 工具调用/函数定义等隐式 prompt 也需计入

14. 输入处理速度（Input TPS）

项目	内容
缩写	Input TPS
单位	tokens/s
定义	模型处理输入 token 的速度

详细解释：

Input TPS 反映 Prefill 阶段的吞吐能力。由于 Prefill 是计算密集型（所有输入 token 并行处理），Input TPS 通常远高于 Output TPS。

计算方式：

Plain Text

```
1 Input TPS = 输入 token 数 / Prefill 耗时
```

实际参考值（Llama-3-8B，单卡 RTX 4090）：

- Input TPS $\approx$ 5,000-15,000 tokens/s
- 远高于 Output TPS（65-80 tokens/s FP16）

影响因素：

- GPU 算力（计算密集型）
- 输入长度（越长越接近峰值算力利用率）
- Flash Attention 等优化

15. 输出 Token 数

项目	内容
单位	tokens

定义	模型生成的文本长度（completion 长度）
----	--------------------------

详细解释：

输出 token 数直接影响：

- Decode 总耗时（线性关系）
- TTLT / ITL（正相关）

影响因素：

- 用户请求的 max_tokens 设置
- 模型的停止条件（EOS token、stop words）
- 采样参数（temperature、top_p 影响生成意愿）
- 思维链模式（thinking 模式会大幅增加输出 token 数）

16. 输出生成速度（Output TPS）

项目	内容
缩写	Output TPS
单位	tokens/s
定义	模型生成输出 token 的速度

详细解释：

Output TPS 是用户最直观感知的"打字速度"，也是推理引擎性能的核心指标。

计算方式：

Plain Text

1

Output TPS = 输出 token 数 / Decode 总时间

2

Output TPS = 1000 / TOPT

实际参考值（Llama-3-8B, RTX 4090）：

精度/量化	Output TPS	说明
-------	------------	----

FP16	65-80 t/s	基准
INT8	100-130 t/s	1.5x 加速
AWQ INT4 (无 Marlin)	100-140 t/s	1.5-2x
AWQ INT4 + Marlin	500-741 t/s	8-10x

影响因素：

- **显存带宽**：Decode 的核心瓶颈，带宽越高 TPS 越高
- **量化精度**：减少权重大小 → 减少访存量 → 提高 TPS
- **模型参数量**：参数越多 → 权重越大 → 访存越多 → TPS 越低
- **上下文长度**：KV Cache 越大 → 注意力访存越多 → TPS 略降
- **Batch Size**：多请求共享权重读取 → 单卡总 TPS 提升，但单请求 TPS 可能下降

17. 每秒查询数（QPS, Queries Per Second）

项目	内容
缩写	QPS
单位	queries/s
定义	系统每秒处理的请求数

详细解释：

QPS 是衡量系统承载力的最直观指标。一个"查询"（query）指一次完整的推理请求（含输入和输出）。

计算方式：

Plain Text

1 QPS = 成功请求数 / 统计时间窗口

与并发数的关系：

Plain Text

1 $QPS = \text{并发数} / \text{平均请求耗时}$

例如：平均每个请求 2 秒，10 个并发 → $QPS = 10 / 2 = 5$

实际参考值：

- 单卡 RTX 4090 (8B FP16)：QPS ≈ 2-5（取决于输入输出长度）
- 4×A100 集群 (8B)：QPS ≈ 20-50
- vLLM 优化后可提升 2-4x

影响因素：

- 单请求耗时（TTFT + Decode 时间）
- GPU 数量和型号
- 调度策略（Continuous Batching vs Static Batching）
- 显存容量（决定最大并发数）

18. 并发数 (Concurrency)

项目	内容
缩写	Concurrency
单位	个
定义	系统能同时处理的请求数

详细解释：

并发数反映系统的并行处理能力。大模型推理中，并发数受显存容量的硬约束：每个并发请求都需要独立的 KV Cache。

显存约束：

Plain Text

1 $\text{最大并发数} \approx (\text{总显存} - \text{模型权重} - \text{框架开销}) / \text{单请求 KV Cache}$

单请求 KV Cache 估算：

Plain Text

1
$$\text{KV Cache} = 2 \times \text{层数} \times \text{头维度} \times \text{KV头数} \times \text{序列长度} \times \text{精度字节数}$$

例如：Llama-3-8B，FP16，4096 上下文：

Plain Text

1
$$\text{KV Cache} = 2 \times 32 \times 128 \times 8 \times 4096 \times 2 = 4\text{GB (仅 KV Cache!)}$$

并发 vs 吞吐 vs 延迟：

- 并发 ↑ → 系统总吞吐 ↑ → 单请求延迟 ↑
- 存在甜点：吞吐接近峰值、延迟尚可接受

19. 系统总吞吐量（System Throughput）

项目	内容
缩写	System Throughput
单位	tokens/s
定义	多并发下，系统每秒输出的总 token 数（所有请求合计）

详细解释：

系统总吞吐 = 所有并发请求的 Output TPS 之和。这是衡量硬件资源利用效率的终极指标。

计算方式：

Plain Text

1
$$\text{系统总吞吐} = \Sigma(\text{每个请求的 Output TPS}) = \text{QPS} \times \text{平均输出 token 数}$$

与单请求 TPS 的关系：

- 理想情况：系统吞吐 = 并发数 × 单请求 TPS

- 实际情况：系统吞吐 < 并发数 × 单请求 TPS（因为共享资源竞争）
- 吞吐效率比 = 系统吞吐 / (并发数 × 单请求 TPS)

20. 吞吐效率比

项目	内容
单位	无量纲
定义	系统实际总吞吐与理论最大吞吐的比值

详细解释：

Plain Text

1 吞吐效率比 = 系统总吞吐 / (并发数 × 单请求 TPS)

- 理想值 = 1.0（每个并发都保持单请求时的 TPS）
- 实际值通常 0.5-0.9（受资源竞争影响）
- 越接近 1.0 说明调度越高效

低效率的原因：

- KV Cache 显存竞争 → 交换/换入换出
- GPU 算力竞争 → 请求间抢占
- 内存带宽饱和

21. 首 Chunk 延迟

项目	内容
单位	毫秒（ms）
定义	从请求发出到接收到第一个流式 chunk 的时间

详细解释：

流式场景下的 TTFT 等价指标。区别在于：

- TTFT 关注"第一个 token"
- 首 Chunk 延迟关注"第一个 chunk"（可能包含多个 token）

影响因素：与 TTFT 相同，加上 chunk 大小的配置。

22. Chunk 间隔

项目	内容
单位	毫秒（ms）
定义	流式输出中相邻 chunk 的时间间隔

详细解释：

衡量流式输出的流畅度。间隔均匀 → 体验流畅；间隔波动大 → 感知卡顿。

理想情况：chunk 间隔稳定在一个较小值（如 50-100ms）。

糟糕情况：有的 chunk 10ms，有的 500ms，用户感知"一顿一顿的"。

23. Chunk TPS

项目	内容
单位	tokens/s
定义	每个流式 chunk 中的 token 生成速率

详细解释：

流式场景下的单请求吞吐指标。每个 chunk 包含若干 token， $\text{Chunk TPS} = \text{chunk 内 token 数} / \text{chunk 生成时间}$ 。

三、资源效率类指标（15 个）

衡量硬件资源使用效率，直接影响成本和部署可行性。

24. 模型权重显存

项目	内容
----	----

单位	GB
定义	存储模型参数所需的显存

详细解释：

模型权重是最基础的显存占用，与参数量和精度直接相关：

精度	每参数字节数	7B 模型	13B 模型	70B 模型
FP32	4 bytes	28 GB	52 GB	280 GB
FP16/BF16	2 bytes	14 GB	26 GB	140 GB
INT8	1 byte	7 GB	13 GB	70 GB
INT4	0.5 bytes	3.5 GB	6.5 GB	35 GB

注意事项：

- 实际占用略大于理论值（对齐开销、框架元数据）
- 量化后的缩放因子也占显存（INT4 实际 ≈ 0.6 bytes/param）
- 多卡部署时权重需复制到每张卡（TP 模式切分，DP 模式复制）

25. KV Cache 显存

项目	内容
定义	存储注意力键值缓存所需的显存
单位	GB

详细解释：

KV Cache 是大模型推理中常被忽视但至关重要的显存占用项。它存储每一层的 Key 和 Value 矩阵，避免 Decode 阶段重复计算。

计算公式：

Plain Text

1 KV Cache = 2 × 层数 × KV头数 × 头维度 × 序列长度 × 精度字节数 × batch_size

示例（Llama-3-8B，FP16，单请求，不同上下文长度）：

上下文长度	KV Cache 大小
512 tokens	~0.5 GB
2K tokens	~2 GB
4K tokens	~4 GB
8K tokens	~8 GB
32K tokens	~32 GB
128K tokens	~128 GB

关键洞察：对于 70B 模型 + 32K 上下文 + 多并发，KV Cache 可能远超模型权重本身！

优化方向：

- KV Cache 量化（FP8/INT4）
- PagedAttention（vLLM）减少碎片
- 滑动窗口注意力（Mistral）
- KV Cache 淘汰策略（H2O、Scissorhands）
- GQA 减少 KV 头数（Llama-3-8B 用 8 KV 头而非 32 Q 头）

26. 激活值显存

项目	内容
单位	GB
定义	推理过程中间激活值占用的显存

详细解释：

推理时每层的前向传播会产生中间激活值（注意力分数、FFN 中间层等），需要临时显存存储。

与训练的区别：

- 训练时需保存所有激活值用于反向传播 → 显存占用巨大
- 推理时只需当前层的激活值 → 显存占用远小于训练

影响因素：

- Batch size：越大激活值越多
- 序列长度：越长注意力矩阵越大
- 模型结构：FFN 维度、注意力头数

27. 峰值显存

项目	内容
单位	GB
定义	推理过程中的最大显存使用量

详细解释：

峰值显存 = 模型权重 + KV Cache + 激活值 + 框架开销，是部署时的显存需求下限。

部署决策：

Plain Text

1 峰值显存 ≤ GPU 显存容量 → 可部署

2 峰值显存 > GPU 显存容量 → 需要：量化 / 切卡 / CPU 卸载

实际参考值（单请求，4K 上下文）：

模型	FP16 峰值显存	INT4 峰值显存
7B	~18 GB	~7 GB
13B	~32 GB	~12 GB
70B	~160 GB	~55 GB

28. CPU 内存占用

项目	内容
单位	GB

定义	模型运行时占用的 CPU 内存
----	-----------------

详细解释：

CPU 内存占用包括：

- CPU 推理模式下的模型权重
- GPU 推理模式下的数据预处理缓冲区
- Tokenizer 词表和缓存
- 框架运行时开销（PyTorch、vLLM 等）
- CPU 卸载模式下暂存的权重分片

场景差异：

- GPU 推理：CPU 内存 $\approx$ 模型权重的 50-100%（加载缓冲）
- CPU 推理：CPU内存 $\approx$ 模型权重 + KV Cache + 激活值
- CPU+GPU 混合：CPU 内存 $\approx$ 模型权重（卸载部分）

29. 模型加载时间

项目	内容
单位	秒
定义	模型从开始加载到准备就绪所需的时间

详细解释：

模型加载时间包含以下阶段：

1. 磁盘读取模型文件
2. 反序列化权重
3. 权重传输到 GPU（PCIe/CNVLink）
4. KV Cache 预分配
5. 首次推理预热（JIT 编译等）

影响因素：

- 模型文件大小（量化后更小 → 更快）

- 磁盘速度 (SSD vs HDD 差距数倍)
- PCIe 带宽 (PCIe 4.0 x16 $\approx$ 32 GB/s)
- 框架初始化开销

实际参考值 (7B FP16, SSD):

- 从磁盘加载: 5-15 秒
- 从内存缓存加载: 2-5 秒
- safetensors 格式比 PyTorch bin 快 2-5x

30. GPU 计算利用率

项目	内容
缩写	GPU Util
单位	%
定义	推理期间 GPU 计算单元 (SM) 的占用比例

详细解释:

GPU 利用率反映 GPU 计算资源的实际使用情况, 但需要注意:

不同阶段的 GPU 利用率:

- Prefill 阶段: 80-99% (计算密集型, 充分利用)
- Decode 阶段: 5-20% (访存密集型, 大量时间在等数据)

常见误区:

- GPU 利用率低 $\neq$ GPU 闲着! Decode 阶段 GPU 在等显存数据返回, 计算单元空闲但显存控制器繁忙
- 应同时看 GPU 利用率和显存带宽利用率

优化方向:

- Continuous Batching: 将多个请求的 Decode 步合并, 提高计算密度
- 投机解码: 用小模型猜测多个 token, 增加单步计算量
- Tensor Parallelism: 切分计算到多卡

31. 显存带宽利用率

项目	内容
缩写	BW Util
单位	%
定义	实际使用显存带宽 / 理论峰值显存带宽

详细解释：

显存带宽利用率是 Decode 阶段的核心瓶颈指标。Decode 每步只需读取权重和 KV Cache 做极少计算，瓶颈在于"把数据从显存搬到计算单元"的速度。

理论分析：

Plain Text

1 Decode 单步所需带宽 = 模型权重大小 + KV Cache 大小

2 理论最大 TPS = 显存带宽 / 单步所需带宽

示例（Llama-3-8B, FP16, A100 80GB）：

Plain Text

1 单步带宽需求 = 14 GB（权重）+ KV Cache

2 A100 带宽 = 2039 GB/s

3 理论最大 TPS ≈ 2039 / 14 ≈ 145 tokens/s

优化方向：

- 量化减少权重大小 → 带宽需求降低 → TPS 提升
- Marlin 内核优化访存模式
- GQA 减少 KV 头数 → KV Cache 更小

32. 算力利用率（MFU, Model FLOPs Utilization）

项目	内容
----	----

缩写	MFU
单位	%
定义	实际完成的 FLOPs / 硬件理论峰值 FLOPs

详细解释：

MFU 是最严格的计算效率指标，衡量"理论需要多少计算"vs"硬件能提供多少计算"的比值。

计算方式：

Plain Text

1 MFU = (实际推理 FLOPs / 推理时间) / 硬件理论峰值 FLOPS

业界参考水平：

- 优秀：50-60%
- 良好：30-50%
- 一般：15-30%
- 差：< 15%

为什么永远达不到 100%：

- 显存带宽瓶颈（Decode 阶段）
- 通信开销（多卡场景）
- 算子启动延迟
- 内存对齐和填充

33. 硬件算力利用率（HFU, Hardware FLOPs Utilization）

项目	内容
缩写	HFU
单位	%
定义	考虑冗余计算后的实际算力效率

详细解释：

HFU 与 MFU 的区别：

- MFU：分子是"理论需要的 FLOPs"
- HFU：分子是"实际执行的 FLOPs"（含重计算、填充等）

当存在冗余计算（如重计算、padding 填充）时， $HFU > MFU$ 。HFU 更贴近硬件实际忙碌程度，MFU 更贴近算法效率。

34. 推理 FLOPs

项目	内容
单位	GFLOPs
定义	单次推理所需的浮点运算量

详细解释：

计算公式：

Plain Text

1

Prefill FLOPs $\approx 2 \times \text{参数量} \times \text{输入 token 数}$

2

单步 Decode FLOPs $\approx 2 \times \text{参数量}$

3

总 Decode FLOPs $\approx 2 \times \text{参数量} \times \text{输出 token 数}$

4

总推理 FLOPs = Prefill FLOPs + 总 Decode FLOPs

示例（8B 模型，1000 输入 tokens，500 输出 tokens）：

Plain Text

1

$\text{Prefill} = 2 \times 8B \times 1000 = 16,000 \text{ GFLOPs} = 16 \text{ TFLOPs}$

2

$\text{Decode} = 2 \times 8B \times 500 = 8,000 \text{ GFLOPs} = 8 \text{ TFLOPs}$

3

总计 = 24 TFLOPs

35. 能效比 (Tokens/Watt)

项目	内容
----	----

单位	tokens/W
定义	每瓦特电能生成的 token 数

详细解释：

Tokens/Watt 是边缘部署和绿色计算的关键指标，反映"花一度电能产出多少文字"。

影响因素：

- 量化精度 (INT4 > INT8 > FP16)
- 架构效率 (GGUF Q4_1 在 Apple Silicon 上能效优秀)
- GPU 型号 (Ada Lovelace 架构比 Ampere 更省电)
- CPU vs GPU (CPU 在低并发下能效可能更高)

实际意义：

- 数据中心：电费是长期运营的主要成本
- 边缘设备：功耗直接影响电池续航
- 碳排放合规：欧盟等地区要求报告 AI 碳足迹

36. 通信量

项目	内容
单位	GB
定义	多卡/多机推理时节点间的数据传输量

详细解释：

不同并行策略的通信模式：

并行方式	通信模式	通信量/步
张量并行 (TP)	AllReduce	$\sim 2 \times \text{模型权重} / \text{TP度} \times 2$
流水线并行 (PP)	点对点 (P2P)	$\sim \text{激活值大小}$
数据并行 (DP)	无 (独立推理)	0

TP 的通信瓶颈：每步 Decode 需 2 次 AllReduce（1 次 attention + 1 次 FFN），通信量与模型大小成正比。当 TP 度高时，通信可能成为瓶颈。

37. 通信占比

项目	内容
单位	%
定义	通信时间 / 总推理时间

详细解释：

通信占比反映计算与通信的平衡性：

- < 10%：通信影响可忽略
- 10-30%：通信有一定影响，需优化
- 30%：通信严重拖慢推理，需调整并行策略

优化方向：

- 使用 NVLink（900 GB/s）替代 PCIe（64 GB/s）
- 计算通信重叠（在通信的同时做其他计算）
- 降低 TP 度，增加 PP 度
- 使用量化减少通信数据量

38. 磁盘 I/O 速率

项目	内容
单位	MB/s
定义	模型加载时的磁盘读取速度

详细解释：

磁盘 I/O 直接影响冷启动时间和模型加载时间。

参考速率：

- NVMe SSD：3,000-7,000 MB/s

- SATA SSD：500-550 MB/s
- HDD：100-200 MB/s
- 网络存储（NFS）：取决于网络带宽

优化方向：

- 使用 safetensors 格式（支持 mmap 零拷贝加载）
- 模型分片并行加载
- 使用内存缓存（第二次加载更快）
- 使用 NVMe SSD

四、模型质量类指标（10 个）

衡量模型本身的输出质量，与推理速度无关，但决定结果是否可用。

39. 困惑度（PPL, Perplexity）

项目	内容
缩写	PPL
单位	无量纲
定义	模型对文本的预测不确定度，越低越好

详细解释：

困惑度衡量模型"对一段文本有多不意外"：

- PPL = 1：模型完美预测每个词（不可能）
- PPL = 10：平均每个词在 10 个候选中选对（较好）
- PPL = 100：平均每个词在 100 个候选中选对（较差）

计算方式：

Plain Text

1 PPL = exp(交叉熵损失) = exp(-1/N × Σ log P(wi|w1...wi-1))

实际参考值：

- GPT-4 级别：PPL $\approx$ 5-8 （英文 WikiText）
- Llama-3-8B：PPL $\approx$ 5.5-6.5
- 量化后通常增加 0.1-1.0

40. 困惑度偏差 (Δ PPL)

项目	内容
缩写	Δ PPL
单位	无量纲
定义	量化/优化后模型 vs 原始模型的困惑度差值

详细解释：

Δ PPL 是量化质量的核心衡量标准：

- Δ PPL = 0：无损量化（不可能）
- Δ PPL < 0.1：近无损（Q8_0 通常可达到）
- Δ PPL = 0.1-0.5：轻微损失（Q4_K_M 级别）
- Δ PPL > 1.0：明显损失（Q2_K 级别）
- Δ PPL > 5.0：严重损失（IQ1_M 级别）

41. 基准测试得分

项目	内容
单位	分
定义	在 MMLU、HumanEval、GSM8K 等基准上的得分

详细解释：

常见基准测试：

基准	评估能力	满分
MMLU	多学科知识	100
HumanEval	代码生成	100
GSM8K	数学推理	100
HellaSwag	常识推理	100
ARC-Challenge	科学推理	100
TruthfulQA	事实准确性	100
MT-Bench	多轮对话	10
AlpacaEval	指令遵循	100

42. 质量保留率

项目	内容
单位	%
定义	量化后模型在基准测试上保持原始精度的比例

详细解释：

Plain Text

1

质量保留率 = 量化后得分 / 原始得分 × 100%

参考值：

- Q8_0：99%+
- Q6_K：96-99%
- Q5_K_M：93-97%
- Q4_K_M：90-95%
- Q3_K_M：80-88%

- Q2_K: 65-78%

43. 幻觉率

项目	内容
单位	%
定义	生成内容中与事实不符的比例

详细解释：

幻觉是大模型在生产部署中的最大风险之一。常见类型：

- **事实性幻觉：** 编造不存在的事实、人物、论文
- **逻辑性幻觉：** 推理过程中出现逻辑错误
- **来源幻觉：** 引用不存在的来源或数据

评估方法：

- TruthfulQA 基准
- 人工评估
- RAG 场景中的检索-生成一致性检查
- 自一致性（多次采样取一致）

44. 重复率

项目	内容
单位	%
定义	输出中重复内容（n-gram 重复）的比例

详细解释：

重复是量化模型和高温度采样的常见问题：

- 轻微：重复几个词 ("很好很好很好")
- 中度：重复整个短语 ("我认为这个问题很重要，我认为这个问题很重要")

- 严重：陷入无限循环

缓解方法：

- presence_penalty（存在惩罚）：1.0-1.5 是常用值
- frequency_penalty（频率惩罚）
- 降低 temperature
- 重复 n-gram 检测和截断

45. 拒答率

项目	内容
单位	%
定义	模型拒绝回答合理问题的比例

详细解释：

过度安全对齐会导致模型"不敢说话"：

- 合理问题被拒："如何编写 Python 排序算法"（被误判为代码漏洞）
- 文化差异："如何评价某历史事件"（被误判为政治敏感）
- 拒答形式：以"我无法..."、"作为AI..."开头

评估方式：

- XSTest 基准
- 人工评估合理问题的拒答比例

46. 指令遵循率

项目	内容
单位	%
定义	模型正确遵循用户指令的比例

详细解释：

指令遵循衡量模型"听不听话":

- 格式要求: "用 JSON 格式输出" → 是否真的输出 JSON
- 长度限制: "100字以内" → 是否控制在 100 字内
- 风格要求: "用古文风格写" → 是否真的用古文
- 多步指令: "先总结再翻译" → 是否两步都做

评估方法:

- IFEval 基准 (自动评估指令遵循率)
- MT-Bench (人工评估多轮指令遵循)

47. 多语言质量差异

项目	内容
单位	分差
定义	不同语言间的基准得分差距

详细解释:

大多数大模型以英文训练为主, 其他语言质量通常有折扣:

- 英文 → 中文: 通常下降 5-15%
- 英文 → 小语种: 可能下降 20-40%
- Qwen 系列中文表现优秀, Llama 系列英文表现优秀

48. 偏见评分

项目	内容
单位	分
定义	模型输出中社会偏见 (性别/种族等) 的程度

详细解释:

偏见评估关注模型是否在敏感属性上产生歧视性输出:

- 性别偏见：默认医生为男性、护士为女性
- 种族偏见：对不同种族的描述差异
- 年龄偏见：对老年人的刻板印象

评估基准：BBQ（Bias Benchmark for QA）、WinoBias

五、量化与压缩类指标（6 个）

49. 量化精度

项目	内容
单位	bit
定义	量化后每个权重的标称位数

详细解释：

量化精度是量化的"名义值"：

- INT4 = 4-bit
- INT8 = 8-bit
- FP16/BF16 = 16-bit
- FP8 = 8-bit

注意：标称值 $\neq$ 有效比特数，因为混合精度和缩放因子会增加实际大小。

50. 有效比特数

项目	内容
单位	bit
定义	量化后每个权重的平均有效位数

详细解释：

有效比特数考虑了缩放因子、混合精度等额外开销后的实际位数：

量化类型	标称精度	有效比特数	差异原因
Q4_K_M	4 bit	4.80 bit	混合精度（部分 6-bit）
Q4_K_S	4 bit	4.60 bit	混合精度（少量 6-bit）
Q5_K_M	5 bit	5.70 bit	混合精度（部分 6-bit）
AWQ INT4	4 bit	~4.20 bit	缩放因子开销
IQ4_XS	4 bit	4.25 bit	imatrix 索引进开销

51. 压缩比

项目	内容
单位	倍
定义	原始模型大小 / 量化后模型大小

详细解释：

Plain Text

1 压缩比 = FP16 模型大小 / 量化后模型大小

参考值：

- INT8：~2x（理论 2x，实际 1.8-2.0x）
- INT4：~3-3.5x（理论 4x，缩放因子和混合精度拉低）
- IQ2_M：~5-6x
- IQ1_M：~7-8x

52. 模型文件大小

项目	内容
单位	GB
定义	量化后模型文件的实际大小

详细解释：

文件大小直接影响：

- 磁盘占用
- 下载/分发时间
- 加载时间
- 是否能在目标设备上存储

53. 量化校准时间

项目	内容
单位	分钟
定义	制作量化模型所需的时间

详细解释：

不同量化方案的校准时间差异：

方案	校准数据需求	校准时间（7B）	校准时间（70B）
GGUF K-quants	可选（无也行）	~15 分钟	~2 小时
GGUF I-quants	推荐（imatrix）	~30 分钟	~4 小时
AWQ	必须（128-512样本）	~10-30 分钟	~1-3 小时
GPTQ	必须（128-1024样本）	~30-60 分钟	~2-6 小时

54. 量化质量损失

项目	内容
单位	无量纲
定义	量化后 vs 原始精度在各基准上的得分差

详细解释：

质量损失通常与量化精度强相关：

- 8-bit：损失 < 1%，几乎无感
- 4-bit：损失 3-8%，多数场景可接受
- 3-bit：损失 8-20%，质量明显下降
- 2-bit：损失 20-40%，仅特定场景可用
- 1-bit：损失 40%+，仅实验价值

六、服务可用性类指标（10 个）

衡量模型作为服务运行时的稳定性和可靠性。

55. 服务可用性（SLA）

项目	内容
缩写	SLA
单位	%
定义	服务可用时间占总时间的比例

详细解释：

SLA 等级	每月最大不可用时间	适用场景
99.9%	43 分钟	内部工具
99.95%	22 分钟	一般业务
99.99%	4.3 分钟	核心业务

99.999%	26 秒	金融/医疗
---------	------	-------

56. 请求成功率

项目	内容
单位	%
定义	成功请求占总请求的比例

详细解释：

成功率排除了所有失败类型：

- OOM（显存不足）
- 超时
- 内部错误
- 输出格式异常
- 模型崩溃（如无限循环）

目标：生产环境通常要求 > 99.5%

57. 超时率

项目	内容
单位	%
定义	请求超过阈值时间未响应的比例

详细解释：

超时率反映系统在负载下的退化程度。常见超时阈值：

- 实时场景：5-10 秒
- 批量场景：60-300 秒
- 长文档场景：600-1800 秒

58. 错误率

项目	内容
单位	%
定义	请求返回错误响应的比例

详细解释：

常见错误类型：

- OOM：并发过高或上下文过长
- 模型内部错误：数值溢出、NaN
- 格式异常：输出不完整或包含非法字符
- 框架错误：vLLM/TGI 内部异常

59. 冷启动时间

项目	内容
单位	秒
定义	从零开始到首次可响应的时间

详细解释：

冷启动时间 = 模型加载时间 + 权重初始化 + KV Cache 预分配 + 首次推理预热

参考值（7B FP16，SSD，A100）：

- 冷启动：10-30 秒
- 温启动（模型已缓存）：3-10 秒
- 热启动（权重已在 GPU）：1-3 秒

60. 扩缩容延迟

项目	内容
单位	秒

定义	从负载变化到资源调整完成的延迟
----	-----------------

详细解释：

自动扩缩容场景的关键指标。大模型的扩容延迟通常远大于传统微服务（冷启动 + 模型加载需数十秒到数分钟）。

优化方向：

- 预热实例池（保持若干待命实例）
- 模型权重预加载到内存
- 使用更快的存储（NVMe SSD）
- 量化减少加载时间

61. 预热时间

项目	内容
单位	秒
定义	新实例从启动到达到稳定性能的时间

详细解释：

新实例的前几次推理通常较慢，原因：

- JIT 编译（PyTorch Compile、CUDA Kernel 编译）
- GPU 频率爬升（从低功耗模式到高性能模式）
- 缓存冷启动（KV Cache 分配、内存池初始化）

典型预热：前 3-5 次推理后性能稳定。

62. 性能波动率

项目	内容
单位	%
定义	相同请求在不同时间的响应时间变异系数

详细解释：

Plain Text

1 性能波动率 = 响应时间标准差 / 响应时间平均值 × 100%

- < 5%：非常稳定
- 5-15%：可接受
- 15%：不稳定，需排查

波动原因：调度抖动、GPU 降频、KV Cache 换入换出、其他进程干扰

63. 长时间运行退化

项目	内容
单位	%
定义	连续运行 24h+ 后性能下降的比例

详细解释：

长时间运行可能导致：

- 显存碎片化 → 新请求分配失败
- KV Cache 累积（未正确释放）→ 显存不足
- 内存泄漏（框架 Bug）
- GPU 热降频

检测方式：每 N 小时运行固定 benchmark，对比首小时和 N 小时后的性能。

64. OOM 阈值

项目	内容
单位	并发数
定义	触发显存溢出的最小并发数

详细解释：

OOM 阈值是部署时的最大安全并发参考值：

Plain Text

1 OOM 阈值 = (总显存 - 模型权重 - 框架开销) / 单请求峰值显存

实际建议：生产并发数设为 OOM 阈值的 70-80%，留出安全余量。

七、上下文相关指标（6 个）

衡量模型处理不同长度输入的能力。

65. 最大上下文长度

项目	内容
单位	tokens
定义	模型能处理的最大 token 序列长度

详细解释：

最大上下文长度由位置编码和训练长度决定：

模型	训练长度	外推长度
Llama-3-8B	8K	128K（YaRN）
Qwen3-4B	32K	131K（YaRN）
Qwen3-32B	32K	131K
Gemini 1.5 Pro	1M+	—

注意：标称最大长度 ≠ 有效长度，长外推会带来质量下降。

66. 有效上下文长度

项目	内容
----	----

单位	tokens
定义	模型质量无明显退化的实际上下文长度

详细解释：

有效长度通常小于标称最大长度，需通过 **Needle-in-Haystack（大海捞针）** 测试验证：在长文本中随机插入关键信息，测试模型能否检索到。

评估维度：

- 检索准确率 vs 深度（信息位置）和长度（总文本长度）
- 理想结果：在所有位置和长度下准确率 > 95%
- 常见退化：中间位置准确率下降（"Lost in the Middle"现象）

67. TTFT 缩放曲线

项目	内容
单位	无量纲图表
定义	TTFT 随输入长度增长的变化曲线

详细解释：

理想情况：TTFT 与输入长度线性增长。

- 线性增长 → 实现高效
- 超线性增长 → 可能存在算法或实现问题

常见问题：

- 二次增长：注意力计算未优化 → 使用 Flash Attention
- 阶梯式增长：KV Cache 分配策略问题
- 突然飙升：触发显存交换

68. 吞吐缩放曲线

项目	内容
----	----

单位	无量纲图表
定义	Output TPS 随输入长度增长的变化曲线

详细解释：

理论上 Decode 阶段的 TPS 与输入长度无关（每步只读 KV Cache 不重算）。但实际上，KV Cache 增大会导致：

- 注意力访存量增加
- Cache 局部性变差
- TPS 略有下降

参考：4K → 32K 上下文，TPS 可能下降 10-30%。

69. 显存缩放曲线

项目	内容
单位	无量纲图表
定义	峰值显存随上下文长度的变化曲线

详细解释：

显存与上下文长度近似线性关系（KV Cache 占主导）：

Plain Text

1 峰值显存 ≈ 模型权重 + 线性系数 × 上下文长度

实际意义：

- 确定特定 GPU 能支持的最大上下文长度
- 规划多并发时的上下文长度上限
- 评估是否需要 KV Cache 量化

70. 长文本衰减率

项目	内容
单位	%/K tokens
定义	超过训练长度后，模型质量下降的速率

详细解释：

当输入超过训练时的最大长度，位置编码外推会导致质量衰减：

外推方法	衰减特点
无外推	超过训练长度后急剧下降
RoPE 缩放	线性插值，整体质量略降但稳定
YaRN	动态缩放，长距离衰减较缓
NTK-aware	频率感知，衰减最缓

参考值：

- 优秀：< 0.5%/K tokens
- 良好：0.5-2%/K tokens
- 较差：> 2%/K tokens